



**LANDMARK METROPOLITAN
UNIVERSITY INSTITUTE**
THE PRIDE OF AFRICA

FILE HANDLING AND ERROR HANDLING

SE309 – Python Programming with Project

Precious Oben, 2025/2026

LANDMARK DIGITAL LESSONS

(+237) 672 339 570 / 694 990 622 | www.landmark.cm



File handling is used to handle a wide range of operations. It supports reading and writing to files.

Files in this case would be txt and binary files (images and videos).

Python treats files either as binary or text. Each line of code includes a sequence of characters which forms a text file. Each line of a file is terminated with a special character, called **the EOL or End of Line characters** like comma {,} or newline character. It ends the current line and tells the interpreter a new one has begun.

Advantages of file handling

- It allows a variety of operations such as reading, writing, deleting, appending and renaming files.
- It allows working with different file types(eg. txt, binary, csv etc.)
- Python file handling functions work across different platforms for compatibility (e.g. Windows, Mac, Linux).

Disadvantages of File Handling

- Error-prone: File handling operations in Python can be prone to errors, especially if the code is not carefully written or if there are issues with the file system (e.g. file permissions, file locks, etc.).
- Security risks : File handling in Python can also pose security risks, especially if the program accepts user input that can be used to access or modify sensitive files on the system.
- Complexity : File handling in Python can be complex, especially when working with more advanced file formats or operations. Careful attention must be paid to the code to ensure that files are handled properly and securely.
- Performance : File handling operations in Python can be slower than other programming languages, especially when dealing with large files or performing complex operations.

BASIC FILE MODES



Before handling files, we need to understand the different types of modes in which they can be handled:

- **“r”**: Read-only mode. Opens an existing file for a read operation. Raises an error if the file doesn't exist.
- **“w”**: Write mode: Creates a new file or overwrites the existing one.
- **“a”**: Append mode- adds content to the end of the file and does not override existing data and does not read it.
- **“r+”**: Read and write mode. To read and write data into the file. This mode does not override the existing data, but you can modify the data starting from the beginning of the file.
- **“w+”**: Write and read mode. To write and read data. It overwrites the previous file if one exists, it will truncate the file to zero length or create a file if it does not exist.
- **“a+”**: Append and read mode: To append and read data from the file. It won't override existing data.

EXAMPLES



1. Working in Read mode

The basic syntax is below, using the open command and the mode

```
file = open('example.txt', 'r')
```

```
for each in file:
```

```
    print (each)
```

or

```
with open("example.txt", "r") as file:
```

```
    content = file.read()
```

```
    print(content)
```

EXAMPLES



2. Write mode

with `open("new_file.txt", "w")` as file:

```
file.write("This is a new file.\n")
```

```
file.write("It overwrites any existing content.")
```


ERROR HANDLING



Error handling is a process that prevents your program from crashing when something goes wrong. **try-except** blocks are used to catch and handle exceptions.

Exceptions are events that occur during program execution that interrupt the normal flow of the program. They usually signal that something unexpected or erroneous has happened, such as trying to divide by zero or accessing a file that doesn't exist.

Common Built – In Exceptions

1. ZeroDivisionError
2. FileNotFoundError
3. ValueError
4. IndexError
5. TypeError

ERROR HANDLING



Basic Syntax

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Error: Division by zero is not allowed.")
```


ERROR HANDLING



Uses of Exception/ Error Handling

1. Prevents Program Crashes:

Without exception handling, errors cause the program to terminate abruptly. With handling, you can gracefully recover from errors.

2. Handles Unexpected Situations:

For example, in the case of a missing file, instead of crashing, a user-friendly message can be displayed or a new file created.

3. Improves Code Robustness:

Exception handling makes programs more reliable and user-friendly.

ERROR HANDLING



Error handling is a process that prevents your program from crashing when something goes wrong. **try-except** blocks are used to catch and handle exceptions.

Exceptions are events that occur during program execution that interrupt the normal flow of the program. They usually signal that something unexpected or erroneous has happened, such as trying to divide by zero or accessing a file that doesn't exist.

Common Built – In Exceptions

1. ZeroDivisionError
2. FileNotFoundError
3. ValueError
4. IndexError
5. TypeError

LOGGING FOR TRACKING EVENTS



Logging is used to record events, errors and statuses in codes. It is mainly helpful in debugging and monitoring applications.

The logging module in Python provides a flexible framework for recording and tracking events during program execution as it allows developers to save logs to files, display logs on the console, or send them to external systems.

Uses of Logging

1. Logs provide a historical record of events.
2. Logs help categorize events by importance (e.g., INFO, WARNING, ERROR).
3. Debugging is easier when errors happen.
4. Tracks applications' behaviors over time.

LOGGING FOR TRACKING EVENTS



Configuration of the Logging System

```
logging.basicConfig(  
    filename="program.log",  
    level=logging.DEBUG,  
    format="%(asctime)s - %(levelname)s - %(message)s"  
)
```

Log messages can directly be added to codes using the logging functions (debug, info, warning, error, critical) to record events.

```
logging.debug("This is a debug message.")  
logging.info("Program started successfully.")  
logging.warning("This is a warning message.")  
logging.error("An error occurred while processing data.")  
logging.critical("Critical issue! Program shutting down.")
```